SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO4: Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool: Pseudocode Writing Task (Algorithm Design)

Weightage: 20%

QUESTIONS: **Total Marks: 25**

1: XML Parsing Algorithm

Write a detailed **pseudocode algorithm** to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

2: MVC Workflow

Write a **pseudocode representation** of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

3: JSP Request Flow

Write a detailed **pseudocode algorithm** to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

4: Database Retrieval

Write a **pseudocode algorithm** to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set, and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

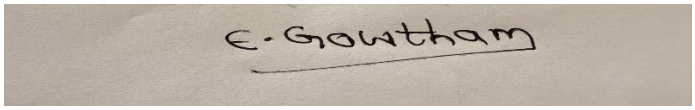
5: PHP Validation

Write a **pseudocode algorithm** for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

Code of Conduct:

I Enamala Gowtham /192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. **XML Parsing Algorithm**

Aim: Parse an XML document, access elements, traverse hierarchical nodes, extract required data, and display it.

XML Parsing Algorithm

Algorithm: Parse XML Document

Input: XML file containing structured user/product data
Output: Extracted XML data displayed in readable format
Step 1: Start.
Step 2: Specify the XML file path.
Step 3: Load the XML document into an XML parser.
Step 4: Check whether the XML document is loaded successfully.

- If loading fails, display **"Error: Unable to load XML file"** and stop.
- Otherwise, continue.

Step 5: Access the root element of the XML document.
Step 6: Identify the required child elements under the root node.
Step 7: Traverse each child node using a loop.
Step 8: For every node:

- Check whether the node contains the required element.
- Extract the element value.
- Extract attributes if required.
- Store the extracted data.

Step 9: If a node contains child nodes, recursively traverse the child nodes to handle the hierarchical structure.
Step 10: Continue traversal until all required nodes are processed.
Step 11: Display the extracted information in a readable format such as:

- ID
- Name
- Email
- Product
- Price

Step 12: Close the XML parser and release resources.
Step 13: Stop.

The answer covers loading, element access, node traversal, data extraction, readable output, and hierarchical XML handling as required.

2. MVC Workflow

Aim: Represent the complete MVC request and response workflow.

MVC Workflow Algorithm

Algorithm: MVC Request Response

Input: User request
Output: Response generated by the View and returned to the user
Step 1: Start.
Step 2: User sends a request to the web application.
Step 3: Web server receives the request.
Step 4: Forward the request to the **Controller**.
Step 5: Controller identifies the requested operation.
Step 6: Controller validates and processes the request parameters.
Step 7: Controller sends the required request data to the **Model**.
Step 8: Model interacts with the database.
Step 9: Model performs the required database operation:

- Establish database connection.
- Execute the required SQL query.
- Retrieve or update data.
- Process the result.

Step 10: Model returns the processed data/result to the Controller.
Step 11: Controller sends the result to the **View**.
Step 12: View uses the received data to generate the required web page/response.
Step 13: View returns the generated response to the Controller/Web Server.
Step 14: Web server sends the response to the user.
Step 15: User views the generated result.
Step 16: Stop.

This directly follows the required sequence: **User → Controller → Model → Database → Model → Controller → View → User**.

3. JSP Request Flow

JSP Request–Response Flow Algorithm

Algorithm: JSP Request Flow

Input: Client request
Output: Dynamic HTML response sent to the client
Step 1: Start.
Step 2: Client sends an HTTP request for a JSP page.
Step 3: Web server receives the request.
Step 4: Server identifies the requested JSP page.
Step 5: Check whether the JSP page has already been compiled.

- If not compiled, translate the JSP page into a Servlet.
- Compile the generated Servlet.

Step 6: Execute the JSP/Servlet.
Step 7: JSP/Servlet processes the request parameters.
Step 8: If backend processing is required, forward the request to the appropriate Servlet or backend component.
Step 9: Backend component establishes a connection with the database if required.
Step 10: Execute the required database operation and retrieve the result.
Step 11: Return the retrieved data to the JSP/Servlet.
Step 12: JSP uses the data to generate dynamic HTML content.
Step 13: Server sends the generated HTML response to the client.
Step 14: Client receives and displays the dynamic web page.
Step 15: Stop.

This includes client initiation, server/JSP processing, Servlet/backend interaction, database interaction, dynamic content generation, and response delivery.

4. **Database Retrieval**

Database Retrieval Algorithm

Algorithm: Retrieve Data from Database

Input: Database details and SQL query

Output: Retrieved records displayed to the user
Step 1: Start.
Step 2: Specify database connection details:
- Database URL
- Username
- Password

Step 3: Establish a connection with the database.
Step 4: Check whether the connection is successful.
- If connection fails, display **"Database connection failed"** and stop.
- Otherwise, continue.

Step 5: Create a database statement/query object.
Step 6: Execute the required SQL SELECT query.
Step 7: Store the returned records in a result set.
Step 8: Check whether the result set contains records.
- If no records are found, display **"No records found"**.
- Otherwise, continue.

Step 9: Traverse the result set using a loop.
Step 10: For each record:
- Retrieve the required column values.
- Store/process the values.
- Display the retrieved data.

Step 11: Continue until all records have been processed.
Step 12: Close the result set.
Step 13: Close the statement/query object.
Step 14: Close the database connection.
Step 15: Stop.

This follows the question's required sequence of **connection → SQL query → result set → processing → display**.

5. **PHP Validation**

PHP User Input Validation Algorithm

Algorithm: Validate User Input

Input: User-entered form data
Output: Validated form submission or appropriate error messages
Step 1: Start.
Step 2: Receive the submitted form data.
Step 3: Initialize an empty error list.
Step 4: Read the required fields such as:

- Name/Username
- Email
- Password

Step 5: Check whether any required field is empty.

- If a field is empty, add an appropriate error message.
- Otherwise, continue validation.

Step 6: Validate the email format.

- If the email format is invalid, add **"Invalid email format"**.
- Otherwise, continue.

Step 7: Validate the password.

- Check whether the password satisfies the required length/format.
- If invalid, add **"Invalid password"**.

Step 8: Check the error list.
Step 9: If errors exist:

- Display the appropriate error messages.
- Prevent form submission.

Step 10: Otherwise:

- Accept the submitted data.
- Process/store the validated data.
- Allow form submission.

Step 11: Stop.

This handles empty fields, email/password format validation, error messages, and submission only when all validations succeed.